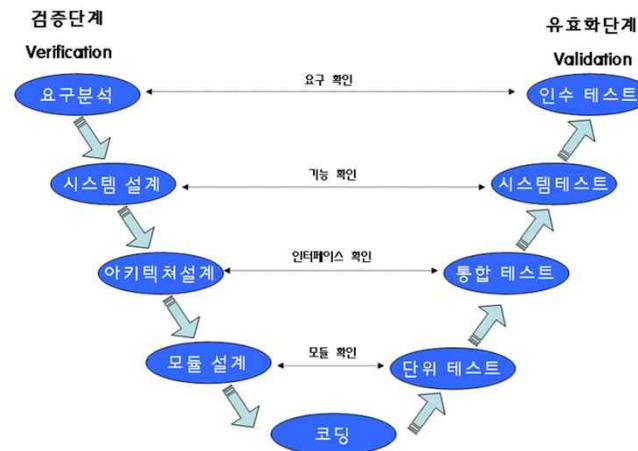


2D 단일 요격-타겟 PN 유도

- 2차원 환경에서 단일 요격자와 단일 타겟의 상태를 기반으로 PN(Proportional Navigation) 유도법칙을 적용하고, 요격자의 유도 명령을 생성·검증할 수 있는 MATLAB 기반 소프트웨어를 개발
- 소프트웨어 개발 생명 주기 모델 - V 모델



- Missile State
 - $[x_M, y_M, \psi_M]^T$
- Environment
 - 2D
- Guidance law
 - Proportional Navigation
- Control Input
 - ψ_{cmd} or ϕ_{cmd}



2D 단일 요격-타겟 PN 유도

- Step 1. 요구사항 분석 – 시스템 범위 및 요구사항

- 시스템 범위
 - 2차원 평면 환경
 - 단일 요격자 / 단일 타겟
 - PN 유도법칙
 - 초기조건 및 시뮬레이션 조건 설정
 - 요격 시뮬레이션 및 결과 분석

ID	요구사항
REQ-001	시스템은 2차원 환경에서 요격자와 타겟의 운동을 모사해야 한다.
REQ-002	시스템은 요격자와 타겟의 초기조건을 설정할 수 있어야 한다.
REQ-003	시스템은 PN 유도법칙을 이용하여 요격 명령을 생성해야 한다.
REQ-004	시스템은 설정된 조건에 따라 요격 시뮬레이션을 수행해야 한다.
REQ-005	시스템은 요격 궤적과 요격 결과를 제공해야 한다.

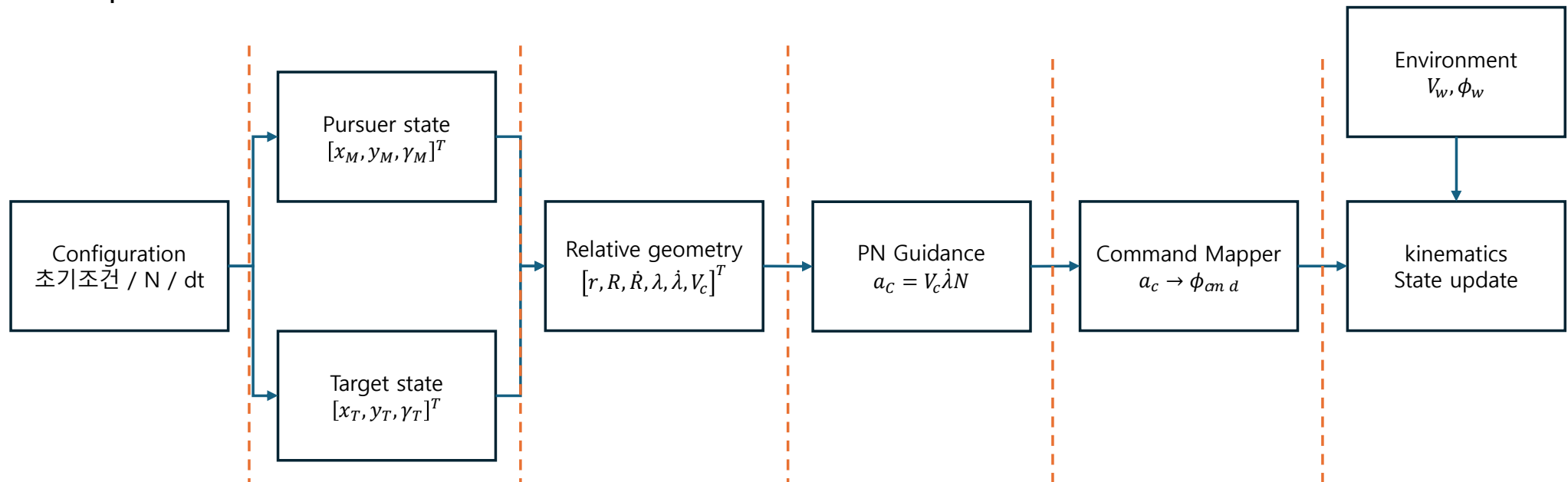
2D 단일 요격-타겟 PN 유도

Step 2. 시스템 설계 - 시스템 기능 구성

기능	역할
조건 설정	초기 상태, 속도, PN 계수, 시뮬레이션 및 환경조건 설정
상대운동 계산	요격자와 타겟의 상대적인 기하관계 계산
유도 명령 생성	상대운동 정보를 이용하여 PN 유도 명령 생성
운동 계산	유도 명령에 따른 요격자와 타겟의 운동 계산
시뮬레이션 수행	시간 진행에 따라 전체 시스템 반복 수행
결과 평가	요격 여부 및 요격 성능 계산
결과 출력	궤적 및 시뮬레이션 결과 표시

2D 단일 요격-타겟 PN 유도

Step 3. 소프트웨어 아키텍처 설계



구분	내용
Input	요격자/타겟 초기조건, 비행조건, PN 설정값, 환경조건, 시뮬레이션 조건
Output	요격자/타겟 궤적, 상대거리, 유도 결과, 요격 여부 및 성능

2D 단일 요격-타겟 PN 유도

Step 3. 소프트웨어 아키텍처 설계 - 모듈 구성

모듈	책임
Configuration	시뮬레이션, 비행체, 유도 및 환경 파라미터 관리
Relative geometry	요격자-타겟 간 상대운동 정보 생성
PN guidance	상대운동 정보로부터 PN 유도 명령 생성
Command mapper	유도 명령을 비행체 제어 명령으로 변환
Kinematics	비행체의 운동 및 상태 변화 계산
Simulation manager	시뮬레이션 순서와 시간 진행 관리
Visualization	궤적 및 결과 시각화

2D 단일 요격-타겟 PN 유도

Step 3. 소프트웨어 아키텍처 설계 – 데이터 구조 정의

Configuration

- Simulation
- Environment
- Pursuer
- Target
- Guidance

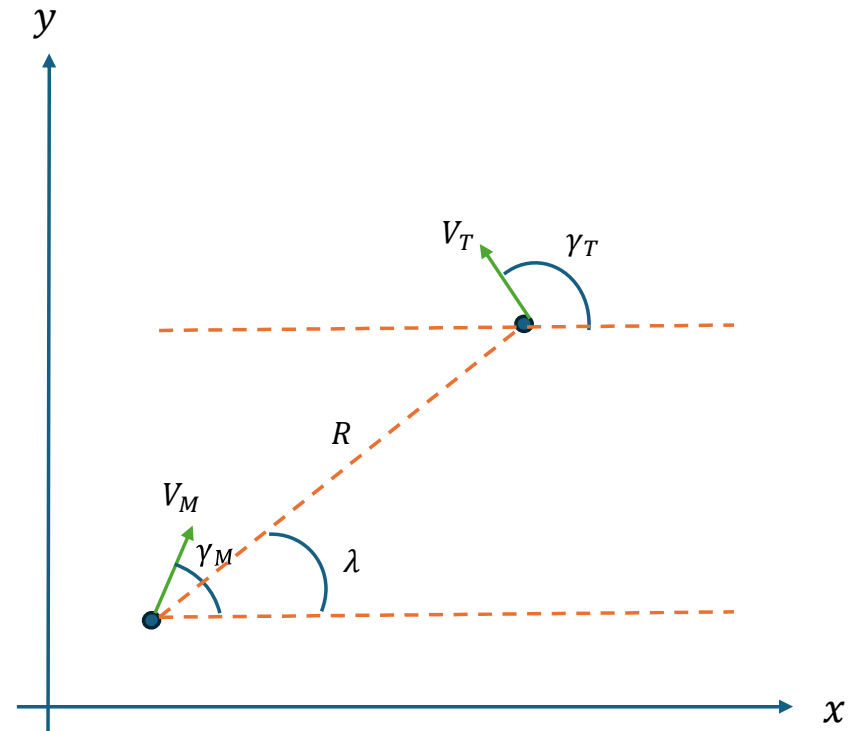
Relative Geometry

- Relative position
- Range
- Range rate
- LoS angle
- LoS rate
- Closing velocity

2D 단일 요격-타겟 PN 유도

Step 3. 소프트웨어 아키텍처 설계 - 좌표계 규약

- Position : $[x, y]$ [m]
- Heading : γ [rad]
- Speed : V [m/s]
- Bank angle : ϕ [rad]
- Acceleration : a [m/s²]
- *Cartesian coordinate*
 - $\dot{x} = V \cos \psi + V_w \cos \psi_w$
 - $\dot{y} = V \sin \psi + V_w \sin \psi_w$
 - $\dot{\psi} = \frac{g \tan \phi}{V}$



2D 단일 요격-타겟 PN 유도

Step 4. 모듈 설계 – 변수 정의

1. 변수 (Variables)

- **형식:** camelCase
- 소문자로 시작
- 여러 단어 사용 시 두 번째 단어부터 첫 글자를 대문자로 작성
- 언더바(_) 사용 지양
- 넓은 범위에서 사용되는 변수는 의미가 명확한 이름 사용
- 좁은 범위에서만 사용되는 변수는 짧은 이름 사용 가능
- 개수를 나타내는 변수는 n 접두사 사용
- 반복문 인덱스는 i + 대상 이름 사용
- Boolean 변수는 is, has, can 등의 긍정형 접두사 사용
- 의미를 알기 어려운 축약어 사용 지양

변수명	역할
time	현재 시뮬레이션 시간
timeStep	시뮬레이션 시간 간격
missileState	Missile 현재 상태
targetState	Target 현재 상태
lateralAccelCmd	횡가속도 명령
missileBankCmd	Missile Bank angle 명령
previousRangeRate	이전 Step의 Range rate
nSteps	전체 Simulation step 개수
iStep	현재 Simulation step index
geometryHistory	상대기하 변수의 시간 이력

2D 단일 요격-타겟 PN 유도

Step 4. 모듈 설계 – 변수 정의

2. 상수 (Constants)

- 형식: UPPER_CASE
- 모든 문자를 대문자로 작성
- 여러 단어는 언더바(_)로 구분
- 프로그램 실행 중 변경되지 않는 값에 사용
- 상수의 목적과 의미가 이름에 드러나도록 작성
- 의미 없는 숫자를 직접 사용하는 Magic Number 지양
- 사용자가 변경할 수 있는 Simulation parameter는 상수가 아닌 설정값으로 관리

상수명	역할
MAX_ITERATIONS	최대 반복 횟수
MAX_BANK_ANGLE	최대 허용 Bank angle
MIN_RANGE	최소 허용 거리
NUMERICAL_TOLERANCE	수치 계산 허용 오차

2D 단일 요격-타겟 PN 유도

Step 4. 모듈 설계 – 변수 정의

3. 구조체 (Structures)

- **형식:** PascalCase
- 구조체 이름은 대문자로 시작
- 구조체 내부 Field는 camelCase 사용
- 서로 관련된 데이터를 하나의 구조체로 그룹화
- 데이터 성격이 다르면 하위 구조체로 구분
- 지나치게 깊은 계층 구조는 지양
- 구조체 및 Field 이름만으로 저장되는 데이터의 의미를 알 수 있도록 작성

구조체명	역할
Config	전체 시뮬레이션 설정 관리
Config.simulation	시뮬레이션 실행 조건
Config.missile	Missile 설정값
Config.target	Target 설정값
Config.environment	환경조건
Config.guidance	유도법칙 파라미터
Missile	현재 Missile 상태 및 비행조건
Target	현재 Target 상태 및 비행조건
Geometry	상대기하 계산 결과
Geometry.range	상대거리 R
Geometry.rangeRate	거리 변화율 $\dot{R}$
Geometry.closingSpeed	Closing speed V_c
Geometry.losAngle	LOS angle λ
Geometry.losRate	LOS rate $\dot{\lambda}$

2D 단일 요격-타겟 PN 유도

Step 4. 모듈 설계 – 변수 정의

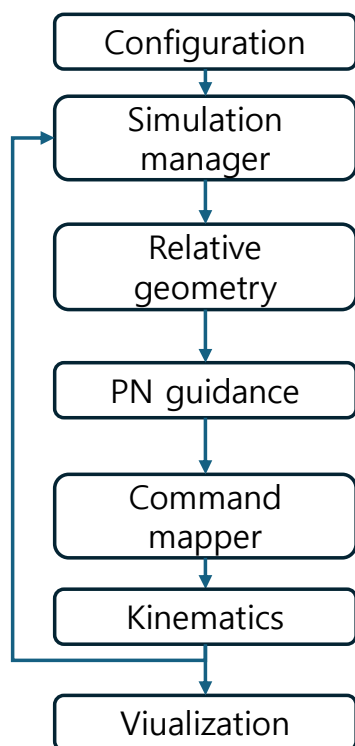
4. 함수 (Functions)

- **형식:** camelCase
- 소문자로 시작
- 함수가 수행하는 **동작을 나타내는 동사**로 시작
- 계산 함수: compute...
- 초기화 함수: initialize...
- 값 획득 함수: get...
- 값 설정 함수: set...
- Boolean 반환 함수: is...
- 의미를 알기 어려운 축약어 사용 지양
- 하나의 함수는 가능한 한 하나의 명확한 기능 수행

함수명	역할
initializeConfiguration	초기조건 및 파라미터 설정
computeRelativeGeometry	Missile-Target 상대기하 계산
computePNGuidance	PN 횡가속도 명령 계산
computeBankCommand	횡가속도를 Bank angle 명령으로 변환
computeKinematics	운동학 상태 미분 계산
visualizeSimulation	시뮬레이션 결과 시각화

2D 단일 요격-타겟 PN 유도

Step 4. 모듈 설계

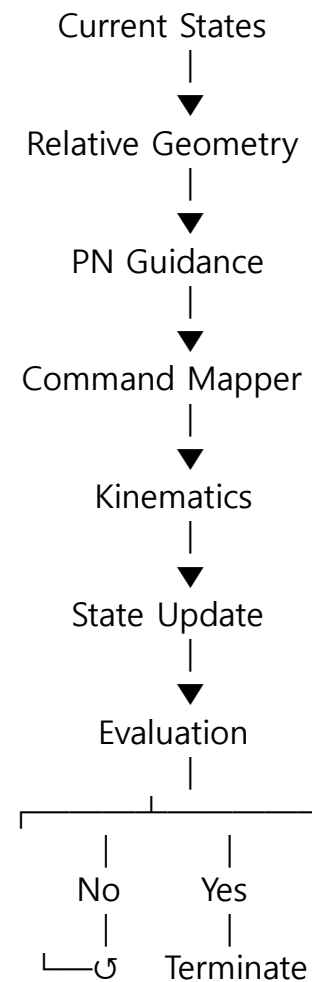
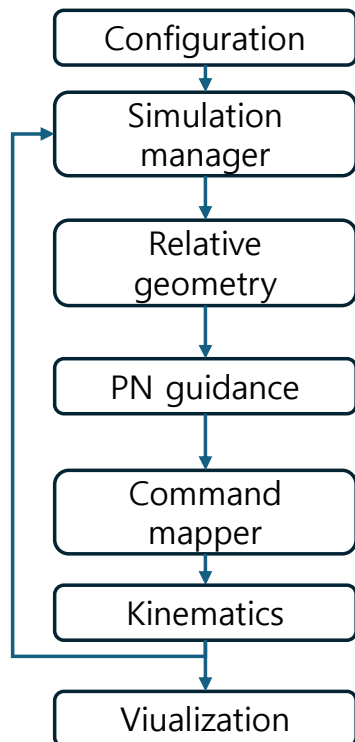


Relative geometry

구분	내용
Input	$X_M = [x_M, y_M, \psi_M]^T, X_T = [x_T, y_T, \psi_T]^T, V_M, V_T$
Relative Position	$\Delta x = x_T - x_M, \Delta y = y_T - y_M$
Range	$R = \sqrt{\Delta x^2 + \Delta y^2}$
LOS Angle	$\lambda = \tan^{-1} \left(\frac{\Delta y}{\Delta x} \right)$
Relative Velocity	$v_{rx} = V_T \cos \psi_T - V_M \cos \psi_M$ $v_{ry} = V_T \sin \psi_T - V_M \sin \psi_M$
Range Rate	$\dot{R} = \frac{\Delta x v_{rx} + \Delta y v_{ry}}{R}$
LOS Rate	$\dot{\lambda} = \frac{\Delta x v_{ry} - \Delta y v_{rx}}{R^2}$
Closing Velocity	$V_c = -\dot{R}$
Output	$R, \lambda, \dot{R}, \dot{\lambda}, V_c$

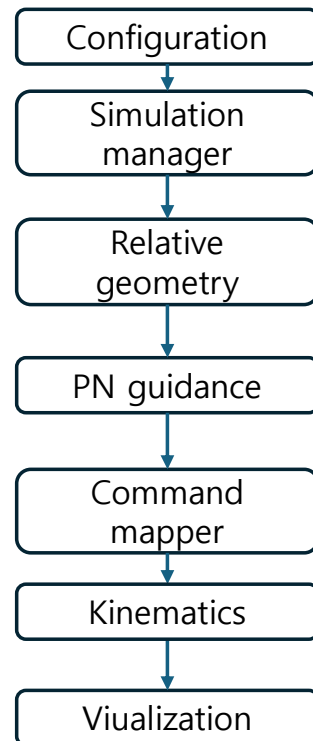
2D 단일 요격-타겟 PN 유도

Step 4. 모듈 설계 – 시뮬레이션 매니저



2D 단일 요격-타겟 PN 유도

Step 4. 모듈 설계



PN guidance

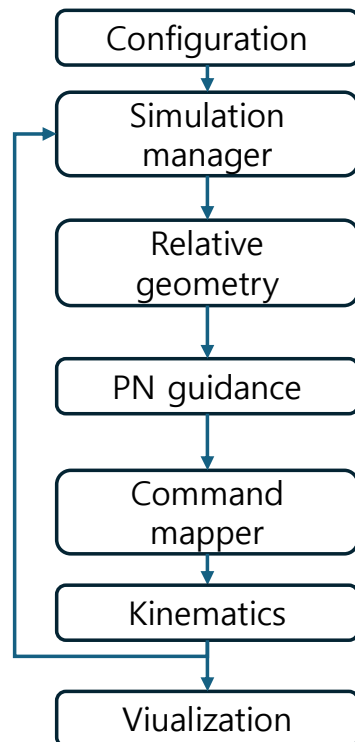
구분	내용
Input	$V_c, \dot{\lambda}, N$
Algorithm	$a_c = N V_c \dot{\lambda}$
Output	<i>Lateral acceleration command and $(a_c, [\frac{m}{s^2}])$</i>

Command mapper

구분	내용
Input	a_c, g
Relationship	$a_c = g \tan \phi_{cmd}$
Algorithm	$\phi_{cmd} = \tan^{-1} \left(\frac{a_c}{g} \right)$
Output	<i>Bank angle command and $(\phi_{cmd}, [rad])$</i>

2D 단일 요격-타겟 PN 유도

Step 4. 모듈 설계 – 상대 지오메트리

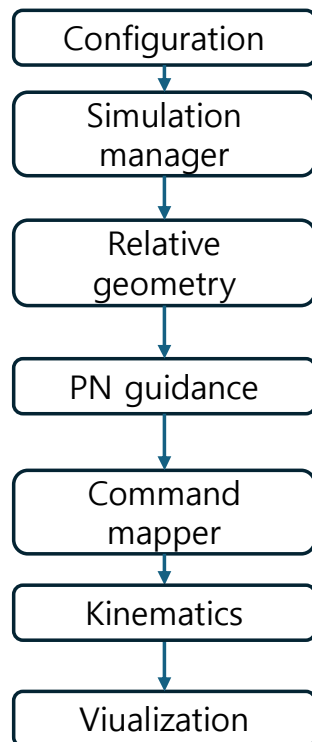


Kinematics

구분	내용
Input	$X = [x, y, \psi]^T, V, \phi, V_w, \psi_w, g$
X-direction	$\dot{x} = V \cos \psi + V_w \cos \psi_w$
Y-direction	$\dot{y} = V \sin \psi + V_w \sin \psi_w$
Heading	$\dot{\psi} = \frac{g \tan \phi}{V}$
Pursuer Input	$\phi = \phi_{cmd}$
Target Input	$\phi_T = 0$ for non-maneuvering target
Output	$\dot{X} = [\dot{x}, \dot{y}, \dot{\psi}]^T$

2D 단일 요격-타겟 PN 유도

Step 4. 모듈 설계



Visualization

구분	내용
Input	저장된 시뮬레이션 상태 및 유도 데이터
Trajectory	요격자 / 타겟 2D 궤적
Relative Motion	$R(t), \dot{\lambda}(t)$
Guidance Command	$a_c(t)$
Control Command	$\phi_{cmd}(t)$
Output	시뮬레이션 결과 그래프

2D 단일 요격-타겟 PN 유도

Step 5. 코딩

파일 명

구현 모듈	MATLAB 파일
Configuration	initializeConfiguration.m
Simulation Manager	main.m
Relative Geometry	computeRelativeGeometry.m
PN Guidance	computePNGuidance.m
Command Mapper	computeBankCommand.m
Kinematics	computeKinematics.m
Visualization	visualizeSimulation.m

2D 단일 요격-타겟 PN 유도

Step 5. Test – 단위 테스트

- $\text{phi_cmd} = 0$